

# Topic 7.5

## 7.5: Practical Example of Private Derivation

The following code fragment illustrates why, in this case, private derivation is needed, as opposed to public derivation.

```
class Graph
{
    public:
    Graph();
    // Return 0 for failure, 1 for success.
    int    AddNode( ... );
    int    AddEdge( ... );
    int    RemoveNode( ... );
    int    RemoveEdge( ... );
    int    PathBetween(src_node, second_node);
    void    print();
}

// Tree is a private derivation from Graph.
class Tree : private Graph
{
    public:
        // Return 0 for failure, 1 for success.
        // These functions must ensure that
        // the object remains a tree.
    int    AddNode( ... );
    int    AddEdge( ... );
    int    RemoveNode( ... );
    int    RemoveEdge( ... );

    int    PathBetween(src_node, second_node);
    void    print() { this->Graph::print(); };
}

void main()
{
    Graph    g;
    Tree    t;

    g.AddEdge(...);
    t.AddEdge(...);

    // The following member function invocation is
    // disallowed because of the private derivation.
    // If it were allowed (via public derivation),
    // it might be possible to add edges to t or
    // remove edges from t in such a way that it is
    // no longer a tree, but still an ordinary
    // graph. It is the private derivation that
    // prevents this access.

    // t.Graph::AddEdge(...);
}
```

**Topic 7.6 (<https://onlinelearning.berkeley.edu/courses/2291660/pages/topic-7-dot-6>)**